

Report: Simple C Project with Makefile

Introduction

This document provides an overview of a simple C project, including its source code, a Makefile, and the steps to implement various Makefile targets for building and managing the project.

Source Code

Below is the main C program (`main.c`), which initializes two integers, calculates their sum, and prints the result alongside a greeting.

```
main.c

#include <stdio.h>
#include <stdlib.h>

// #define MY_GLOBAL_VARIABLE 120

int main(void)
{
    // int c = MY_GLOBAL_VARIABLE;
    // puts("Hello");

    int val_two = 0;
    int val_one = 0;
    int result = 0;

    result = val_one + val_two;

    printf("\n%d + %d = %d\n\n", val_one, val_two, result);
    char hellow[] = "hellow";
    printf("%s\n", hellow);

    return EXIT_SUCCESS;
}
```

Makefile

The project uses a Makefile to simplify the build process and manage multiple targets. The Makefile includes the following targets:

1. all: Builds the application for the system's default architecture.
2. build: Compiles the C application into an executable.
3. x86: Builds a 32-bit version of the application.

4. x86_64: Builds a 64-bit version of the application.
5. asm: Generates assembly output in Intel format.
6. clean: Removes all build artifacts.
7. doc: Generates a PDF documentation file from report.md.
8. debug: Runs the application in gdb for debugging.

Makefile

```
CFLAGS = -Wall -Iinclude -IUnity/src -fprofile-arcs -ftest-coverage
LDFLAGS = -L. -fprofile-arcs -ftest-coverage

# Input files
SRC = main.c
OBJ = $(SRC:.c=.o)
EXECUTABLE = app

# Default target
.PHONY: all build x86 x86_64 asm clean doc debug

# Targets
all: build

build: $(EXECUTABLE)

$(EXECUTABLE): $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

x86:
    $(MAKE) CFLAGS="$(CFLAGS) -m32" LDFLAGS="$(LDFLAGS) -m32" build

x86_64:
    $(MAKE) CFLAGS="$(CFLAGS) -m64" LDFLAGS="$(LDFLAGS) -m64" build

asm: $(SRC)
    $(CC) $(CFLAGS) -S -masm=intel $< -o $(SRC:.c=.s)

clean:
    rm -f $(OBJ) $(EXECUTABLE) *.s

doc:
    @echo "Generating documentation..."
    pandoc -o report.pdf report.md || echo "Install pandoc to generate documentation."
```

```
debug: build
      gdb ./${EXECUTABLE}
```

Steps to Build and Run the Project

1. Build the Application: `make build`
2. Generate a 32-bit Application: `make x86`
3. Generate a 64-bit Application: `make x86_64`
4. Generate Assembly Output: `make asm`
5. Clean Build Artifacts: `make clean`
6. Generate Documentation: `make doc`
7. Debug the Application: `make debug`

Conclusion

This project demonstrates the use of a Makefile to streamline compilation, cross-platform builds, and debugging. It provides a foundation for larger and more complex C projects.